

---

#### APPENDIX A: REFERENCE GUIDE FOR PYQT5

PyQt is a Python binding for the Qt application framework maintained by Riverbank Computing Limited. **Abinding** is an application programming interface (API) that provides the code to allow a programming language to use other libraries not native to that language. Qt is a set of C++ libraries and development tools, providing access to networking, threads, SQL databases, OpenGL and other graphics tools, XML, GUI development, and other features. This chapter focuses only on PyQt5, because as of this writing, PyQt4 is no longer supported.

Appendix A contains a reference for some of the tools, modules, and classes learned throughout this book, including

- Installing PyQt5 and Qt Designer
- A review for PyQt modules and classes
- An overview of Qt Style Sheets

More information about Riverbank Computing Limited and PyQt5 can be found at the following link:

<https://riverbankcomputing.com/software/pyqt/intro>

#### INSTALLING PYQT5 AND QT DESIGNER

Before beginning, make sure Python 3 is already installed on your system. If you have not already installed Python, or are not sure if you already have it installed, please check out Appendix B.

Also, the SIP binding generator is a tool used for creating bindings that allow Python to access the C++ classes. If you choose to download PyQt5 from the Python Package Index (PyPI) repository, then the `sip` module will automatically be downloaded, as well.

The following subsections focus on downloading PyQt5 from PyPI using `pip3`, which only operates on Python 3 environments.

#### GETTING PYQT FOR WINDOWS

To install PyQt5, enter the following command into the Command terminal:

```
pip3 install pyqt5
```

You can check to make sure PyQt downloaded properly by opening up the Python 3 interpreter and entering the following command:

```
>>> import PyQt5.QtWidgets
```

If no errors are returned, then the next step is to install Qt Designer. To get Qt Designer, enter the following line into the command line:

```
pip3 install pyqt5-tools
```

Next, we need to locate the Qt Designer executable. You will need to look for the site-packages folder inside the main Python directory. The path should look something similar to the following path:

```
C:\Users\your_info\Local\Programs\Python3_directory\Lib\site-packages\pyqt5_tools
```

Once you have located the executable file labeled `designer`, create a shortcut so you will be able to easily access it for next time.

#### GETTING PYQT FOR MACOS

Make sure that you already have Xcode downloaded. If not, you can get it from the App Store. Similar to Windows, to download PyQt5, enter the following command into the Terminal:

```
pip3 install pyqt5
```

If you are using Homebrew, then you can use the following line instead:

```
brew install pyqt5
```

Next, check to make sure PyQt downloaded properly by opening up the Python 3 interpreter and entering the following command:

```
>>> import PyQt5.QtWidgets
```

If no errors appear, then the next step is to install Qt Designer. For MacOS, the process to download Qt Designer is not as simple. There is no `pyqt5-tools` wheel compatible with MacOS. Therefore, your options are either to download Qt from [www.qt.io/download](http://www.qt.io/download) or download a stand-alone version that is thankfully provided by Michael Herrmann at

<https://build-system.fman.io/qt-designer-download>

If you downloaded the stand-alone version, once the file finishes downloading, add the Qt Designer file to your Applications folder and open it up to get started.

#### GETTING PYQT FOR LINUX (UBUNTU)

For Ubuntu, enter the following command into the shell:

```
sudo apt-get install python-pyqt5
```

Open the Python 3 interpreter and enter the following statement to make sure that PyQt5 is installed properly:

```
>>> import PyQt5.QtWidgets
```

If no errors are returned, then we can install Qt Designer:

```
sudo apt-get install qttools5-dev-tools
```

Next, you need to locate Qt Designer, `designer` , in the following path:

```
/usr/lib/x86_64-linux-gnu/qt5/bin/
```

Finally, make a shortcut for Qt Designer so that you will be able to easily locate the application next time.

OTHER METHODS FOR GETTING PYQT

There are several other ways to install PyQt. One option is to build and install from source. If for some reason you must build from source, check out the following link for the PyQt reference guide:

[www.riverbankcomputing.com/static/Docs/PyQt5/](http://www.riverbankcomputing.com/static/Docs/PyQt5/)

Another method is to download PyQt through the Anaconda distribution . To install the package on any system, run:

```
conda install -c anaconda pyqt
```

Another option is to install PyQt5 in a virtual environment, but that won't be covered here.

SELECTED PYQT5 MODULES

PyQt provides a range of modules that give you access to a wide array of tools, including basic GUI design, 2D and 3D rendering, multimedia content, networking, global positioning, and more. For basic GUI development, you will primarily use the `QtWidgets`, `QtGui`, and `QtCore` modules. Table [A-1](#) lists the modules covered throughout the book, as well as a few extra you should check out.

For a full list of PyQt5's top-level modules, check out the following link:

[www.riverbankcomputing.com/static/Docs/PyQt5/module\\_index.html](http://www.riverbankcomputing.com/static/Docs/PyQt5/module_index.html)

Table A-1 Table of select PyQt modules

| Module Name                 | Description                                                                                                                 |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>QtWidgets</code>      | Provides the widgets and other classes for creating desktop-style UIs.                                                      |
| <code>QtCore</code>         | Contains a variety of extra classes, including the essential non-GUI classes, such as ones for Qt's signal and slot system. |
| <code>QtGui</code>          | Contains classes for 2D graphics and imaging, event handling, and window system integration.                                |
| <code>QtPrintSupport</code> | Provides cross-platform support for configuring and connecting to printers.                                                 |
| <code>QtNetwork</code>      | Provides classes for writing communications protocols using UDP or TCP.                                                     |

| Module Name                      | Description                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>QtMultimedia</code>        | Contains the classes for multimedia content, cameras, and radios.                                   |
| <code>QtMultimediaWidgets</code> | Provides additional classes that increase the functionality of multimedia-related widgets.          |
| <code>QtWebEngineCore</code>     | Contains the core classes used by other WebEngine modules.                                          |
| <code>QtWebEngineWidgets</code>  | Classes that can be used to create a Chromium-based web browser.                                    |
| <code>QtSql</code>               | Provides classes for working with SQL databases.                                                    |
| <code>sip</code>                 | Tools used for creating Python bindings for C++ libraries (which is the language Qt is written in). |
| <code>uic</code>                 | Contains classes used for handling the .ui files created by Qt Designer.                            |

#### SELECTED PYQT CLASSES

There are hundreds of PyQt classes. The following section lists the key classes and widgets that can be found throughout this book. Each subsection either lists tables with commonly used methods and signals, or a link to where you can find more information about the class.

For a list of all the PyQt classes, check out the following link:

[www.riverbankcomputing.com/static/Docs/PyQt5/sip-classes.html](http://www.riverbankcomputing.com/static/Docs/PyQt5/sip-classes.html)

Although it is written for C++, the Qt classes' documentation is generally more detailed. If you want more information about Qt classes, you can also check out

<https://doc.qt.io/qt-5/classes.html>

#### CLASSES FOR BUILDING A GUI WINDOW

With PyQt, you can create a new class that inherits from any of the widget classes. However, for a general GUI application, you will need to create only one instance of `QApplication`, and create a class that inherits from either `QWidget`, `QMainWindow`, or `QDialog` to create the application's window.

#### QAPPLICATION

`QApplication` is responsible for handling the initialization and finalization of widgets in graphical user interfaces. If you are making `QWidget`-based applications, then you will need to create an instance of `QApplication` before creating any other objects related to the GUI.

Some of QApplication’s responsibilities include initializing an application to conform to a user’s desktop settings, event handling, defining the GUI’s style, working with the clipboard, and keeping track of all the application’s windows.

If you are creating an application that does not need a GUI and can be run through the command line, then you should consider using **QCoreApplication** , instead.

**QWIDGET**

The QWidget class is the base class for all of PyQt’s graphical user interface’s objects. A widget created from the QWidget class is able to receive input from the mouse, keyboard, and other events and able to paint itself on the screen. Widgets that are not embedded in a parent widget are considered to be a window complete with a title bar and a frame. The QWidget class is a subclass of QObject and QPaintDevice. Some of QWidget’s more commonly used methods can be found in Table **A-2** .

**Table A-2** Selected methods from QWidget

| Method                                         | Description                                                                                                   |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>addAction (action)</code>                | Add an <code>action</code> to the widget.                                                                     |
| <code>close ()</code>                          | Close the widget.                                                                                             |
| <code>height ()</code>                         | Holds the widget’s height.                                                                                    |
| <code>width ()</code>                          | Holds the widget’s width.                                                                                     |
| <code>move (x, y)</code>                       | Sets the location of the widget to <code>(x ,y )</code> .                                                     |
| <code>rect ()</code>                           | Holds the geometry of the widget minus the frame.                                                             |
| <code>setDisabled (bool)</code>                | If <code>True</code> , the widget is disabled.                                                                |
| <code>setEnabled (bool)</code>                 | If <code>True</code> , the widget is enabled.                                                                 |
| <code>setFont (font)</code>                    | Sets the <code>font</code> of the widget’s text.                                                              |
| <code>setLayout (layout)</code>                | Sets the layout manager for the widget.                                                                       |
| <code>setGeometry (x, y, width, height)</code> | Sets the widget’s location, <code>(x ,y )</code> , and its size, <code>width</code> and <code>height</code> . |
| <code>setStyleSheet (styleSheet)</code>        | Sets the <code>styleSheet</code> for the widget.                                                              |
| <code>setToolTip (text)</code>                 | Sets the widget’s tool tip.                                                                                   |

| Method                        | Description                                                            |
|-------------------------------|------------------------------------------------------------------------|
| <code>repaint()</code>        | Repaints the widget immediately by calling <code>paintEvent()</code> . |
| <code>showFullScreen()</code> | Displays the widget in full screen mode.                               |
| <code>update()</code>         | Updates the widget.                                                    |

EVENT HANDLING

Events are typically caused by users. These can include moving a mouse, pressing a key, or resizing the window. The widgets in an application need to respond to the event caused by the user's actions. The events are typically already handled, but you sometimes may find yourself needing to reimplement event handlers to supply further behavior or content for the widgets. Table [A-3](#) lists a few commonly used event handlers.

Table A-3 Some event handlers used for supplying behavior to QWidget objects

| Event Handler                        | Description                                                                                                                                                            |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>paintEvent()</code>            | Called whenever a widget needs to be repainted.                                                                                                                        |
| <code>resizeEvent()</code>           | Called when a widget has been resized.                                                                                                                                 |
| <code>mousePressEvent()</code>       | Called when a mouse button is pressed while the mouse cursor is inside of the widget. Which mouse button is clicked can be specified in the event.                     |
| <code>mouseReleaseEvent()</code>     | Called when a mouse button is released. A widget that receives this event is dependent on receiving the mouse press event.                                             |
| <code>mouseDoubleClickEvent()</code> | Called when a widget is double-clicked.                                                                                                                                |
| <code>mouseMoveEvent()</code>        | Called when the mouse moves while the button is held down. If <code>setMouseTracking()</code> is <code>True</code> , events are sent even when no buttons are pressed. |
| <code>enterEvent()</code>            | Called when the mouse enters a widget's space.                                                                                                                         |
| <code>leaveEvent()</code>            | Called when the mouse leaves a widget's space.                                                                                                                         |
| <code>keyPressEvent()</code>         | Called when a key is pressed.                                                                                                                                          |

| Event Handler                  | Description                                          |
|--------------------------------|------------------------------------------------------|
| <code>keyReleaseEvent()</code> | Called when a key is released.                       |
| <code>focusInEvent()</code>    | Called when a widget gets the keyboard focus.        |
| <code>focusOutEvent()</code>   | Called when a widget loses the keyboard focus.       |
| <code>closeEvent()</code>      | Called when either a widget or the window is closed. |

QMAINWINDOW

The QMainWindow class provides the framework for building an application, complete with functions for adding a menubar, toolbars, a status bar, and dock widgets. Menu and toolbar items are created using QAction. QMainWindow already has its own layout, to which you must add a central widget as the center area of the application's window. Some of QMainWindow's methods can be seen in Table A-4.

Table A-4 Select methods from QMainWindow

| Method                                       | Description                                                                                    |
|----------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>addDockWidget(area, dockwidget)</code> | Creates <code>adockwidget</code> in the main window in the specified <code>area</code> .       |
| <code>addToolBar(toolbar)</code>             | Creates <code>atoolbar</code> for the main window. An <code>area</code> can also be specified. |
| <code>menuBar()</code>                       | Returns the main window's menubar.                                                             |
| <code>setStatusbar(statusbar)</code>         | Creates the <code>statusbar</code> for the main window.                                        |
| <code>setCentralWidget(widget)</code>        | Sets the window's <code>centralwidget</code> .                                                 |
| <code>setWindowIcon(icon)</code>             | Sets the window's <code>icon</code> .                                                          |
| <code>setWindowTitle(text)</code>            | Sets the window's title.                                                                       |

QDIALOG

Dialog boxes provide a top-level window generally to obtain feedback quickly from a user. QDialogs can be modal or modeless. Modal dialogs are often used when selecting an option in the dialog that will return a value. That value could then be used to save a file, close a document, or cancel an action. Table A-5 lists the QDialog methods useful for creating modal or modeless dialog boxes.

QDialog is the base class for dialog boxes, including QColorDialog, QFileDialog, QFontDialog, QInputDialog, QMessageBox, QProgressDialog, and QErrorMessage.

Table A-5 Select methods for QDialog

| Method   | Description                                                                                                   |
|----------|---------------------------------------------------------------------------------------------------------------|
| accept() | Hides the modal dialog and returns <code>True</code> .                                                        |
| reject() | Hides the modal dialog and returns <code>False</code> .                                                       |
| exec_()  | The dialog is shown as a modal dialog and blocks the user from any further action until the dialog is closed. |
| show()   | The dialog is a modeless dialog, returning control to the user immediately.                                   |

QPAINTER

The QPainter class is responsible for handling drawing in PyQt, being able to draw simple lines and complex shapes onto widgets and other paint devices. QPainter is most commonly used in a QWidget's `paintEvent()` and for handling pixmaps and images. Table A-6 displays some of QPainter's methods for drawing.

Table A-6 Methods selected from QPainter

| Method                                   | Description                                                                                                                                |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| begin(device)                            | Begins painting on the paint device.                                                                                                       |
| end()                                    | Ends painting. Resources used while painting are released.                                                                                 |
| save()                                   | Saves the current painter state. <code>save()</code> must be followed by <code>restore()</code> , which returns the current painter state. |
| drawArc(QRectF, startAngle, spanAngle)   | Draws an arc defined by the <code>QRectF</code> rectangle, <code>startAngle</code> , and <code>spanAngle</code> .                          |
| drawChord(QRectF, startAngle, spanAngle) | Draws a chord defined by the <code>QRectF</code> rectangle, <code>startAngle</code> , and <code>spanAngle</code> .                         |
| drawEllipse(QPointF, x_rad, y_rad)       | Draws an ellipse at <code>QPointF</code> center, with radius <code>x_rad</code> and <code>y_rad</code> .                                   |
| drawLine(x1, y1, x2, y2)                 | Draws a line from point <code>(x1, y1)</code> to <code>(x2, y2)</code> .                                                                   |



| Method                                              | Description                                                                                                                       |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>drawPath(path)</code>                         | Draws a path specified by <code>QPainterPath</code> <code>path</code> .                                                           |
| <code>drawPie(QRectF, startAngle, spanAngle)</code> | Draws a pie defined by the <code>QRectF</code> <code>rectangle</code> , <code>startAngle</code> , and <code>spanAngle</code> .    |
| <code>drawPixmap(x, y, pixmap)</code>               | Draws a <code>pixmap</code> at <code>(x,y)</code> .                                                                               |
| <code>drawPoint(x, y)</code>                        | Draws a point at <code>(x,y)</code> .                                                                                             |
| <code>drawRect(x, y, width, height)</code>          | Draws a rectangle at <code>(x,y)</code> with <code>width</code> and <code>height</code> .                                         |
| <code>drawRoundedRect(QRectF, x_rad, y_rad)</code>  | Draws a rectangle with rounded corners specified by <code>QRectF</code> , with radius <code>x_rad</code> and <code>y_rad</code> . |
| <code>drawText(QPointF, text)</code>                | Draws <code>text</code> at <code>QPointF</code> <code>point</code> .                                                              |
| <code>fillRect(QRectF, brush)</code>                | Fills in a <code>QRectF</code> rectangle with the <code>brush</code> color.                                                       |
| <code>rotate(angle)</code>                          | Rotates the coordinate system clockwise by <code>angle</code> (in degrees).                                                       |
| <code>setBrush(brush)</code>                        | Sets the painter's <code>brush</code> .                                                                                           |
| <code>setPen(pen)</code>                            | Sets the painter's <code>pen</code> .                                                                                             |
| <code>setFont()</code>                              | Sets the painter's <code>font</code> .                                                                                            |

#### LAYOUT MANAGERS

Using PyQt's layout managers makes the process of arranging widgets much easier, compared to manually specifying each widget's size, position, or `resizeEvent()` event handler. Using layout managers is generally a good start for positioning widgets, although you may still need to adjust a widget's size policy, or add stretching or spacing to a layout.

The following classes inherit from the `QLayout` class, which is the base class for layout managers:

- `QBoxLayout` – Arranges child widgets into a row (horizontally), or into a column (vertically)
  - a. `QHBoxLayout` – Arranges widgets horizontally
  - b. `QVBoxLayout` – Arranges widgets vertically
- `QGridLayout` – Orders widgets in a grid of rows and columns

- `QFormLayout` – Lays out widgets into a form-like structure with labels and their associated input widgets

There is also **QStackedLayout** which was not covered in this book. The convenience `QStackedWidget` class is built on top of the `QStackedLayout`. Table [A-7](#) lists commonly used methods from the layout classes.

**Table A-7** Selected methods for the different layout managers

| Method                                                                      | Class                                                 | Description                                                                                        |
|-----------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>addWidget(widget, stretch, alignment)</code>                          | <code>QBoxLayout</code>                               | Add widget to the end of the layout with stretch factor and alignment.                             |
| <code>addWidget(widget, row, column, rowspan, columnSpan, alignment)</code> | <code>QGridLayout</code>                              | Add widget at row, column with (optional) rowspan and columnSpan and alignment.                    |
| <code>addRow(label, field)</code>                                           | <code>QFormLayout</code>                              | Add a new row with given label and field (input widget).                                           |
| <code>addLayout(layout, stretch)</code>                                     | <code>QBoxLayout</code>                               | Adds a layout to the end of the box. Creates a nested layout.                                      |
| <code>addLayout(layout, row, column, alignment)</code>                      | <code>QGridLayout</code>                              | Adds a layout at position (row, column). Creates a nested layout.                                  |
| <code>addSpacing(int)</code>                                                | <code>QGridLayout</code> ,<br><code>QBoxLayout</code> | Adds a non-stretchable area ( <code>aQSpacerItem</code> ) of <code>int</code> value to the layout. |
| <code>addStretch(int)</code>                                                | <code>QBoxLayout</code>                               | Adds a stretchable area ( <code>aQSpacerItem</code> ) of <code>int</code> value to the layout.     |
| <code>setSpacing(int)</code>                                                | <code>QLayout</code>                                  | Sets the space between widgets in the layout.                                                      |

| Method                                      | Class   | Description                                                      |
|---------------------------------------------|---------|------------------------------------------------------------------|
|                                             |         | Inherited from QLayout.                                          |
| setContentMargins(left, top, right, bottom) | QLayout | Sets the left, top, right, and bottom margins around the layout. |

BUTTON WIDGETS

Buttons are one of the main tools used in a GUI for interaction, giving an application feedback about a user’s decisions. Buttons in PyQt can display text or icons and are checkable. The following classes inherit from the base class for button widgets, `QAbstractButton` :

- `QPushButton` – A command button used to tell the computer to perform some action
- `QCheckBox` – Provides an option button that is checkable, and generally used for enabling/disabling features in an application
- `QRadioButton` – Similar to checkboxes, but are mutually exclusive
- `QToolButton` – Typically used in a toolbar, tool buttons provide quick-access buttons for selecting commands or options

For managing and organizing multiple buttons, the `QButtonGroup` class can act as a container for creating exclusive buttons (the default setting). Table [A-8](#) lists some of the more commonly used methods for button widgets.

Table A-8 Selected methods for the different button widgets

| Method                 | Description                                                                  |
|------------------------|------------------------------------------------------------------------------|
| setIcon(icon)          | Sets the widget’s icon .                                                     |
| setText(text)          | Set’s the widget’s text .                                                    |
| setAutoExclusive(bool) | Enables auto-exclusivity for buttons in a group.                             |
| setCheckable(bool)     | Sets whether the button is a toggle button or not.                           |
| setChecked(bool)       | Sets whether the button is checked or not.                                   |
| isChecked()            | Indicates whether the button is checked or not (is setCheckable() is True ). |
| text()                 | Holds the buttons text.                                                      |

The signals of the button widgets are listed in Table [A-9](#) .

Table A-9 Signals for the different button widgets

| Signal                           | Class                        | Description                                               |
|----------------------------------|------------------------------|-----------------------------------------------------------|
| <code>clicked (bool)</code>      | <code>QAbstractButton</code> | Signal emitted when the button is pressed and released.   |
| <code>pressed ()</code>          | <code>QAbstractButton</code> | Emitted when the left mouse button clicks the button.     |
| <code>released ()</code>         | <code>QAbstractButton</code> | Signal emitted when the left mouse button is released.    |
| <code>toggled (bool)</code>      | <code>QAbstractButton</code> | Emitted when a check-able button changes its state.       |
| <code>stateChanged (bool)</code> | <code>QCheckBox</code>       | Emitted when the checkbox's state changes.                |
| <code>triggered (action)</code>  | <code>QToolButton</code>     | Signal emitted when the <code>action</code> is triggered. |

INPUT WIDGETS

There are quite a few widgets that are provided by PyQt for getting input from the user. These widgets provide different means for gathering information, such as text entry, or selecting values with sliders, combo boxes, and spin boxes.

COMBO BOXES

The `QComboBox` class presents a user with a list of selectable options in a compact, drop-down menu. When the combo box is not being interacted with, all items except for the current item selected are hidden from view. Some common methods for `QComboBox` can be found in Table A-10 . The `QFontComboBox` widget is another type of combo box that inherits `QComboBox` and is used for selecting a font family.

Table A-10 Select methods from the `QComboBox` class

| Method                              | Description                                            |
|-------------------------------------|--------------------------------------------------------|
| <code>addItem (text)</code>         | Appends an item to the list with <code>text</code> .   |
| <code>addItems (list (text))</code> | Appends a <code>list</code> of items to the combo box. |
| <code>currentIndex ()</code>        | Holds the index of the currently selected item.        |
| <code>currentText ()</code>         | Holds the text of the currently selected item.         |

| Method                                | Description                                                                        |
|---------------------------------------|------------------------------------------------------------------------------------|
| <code>insertItem(index, text)</code>  | Inserts the <code>text</code> into the combo box at the given <code>index</code> . |
| <code>setItemText(index, text)</code> | Sets the <code>text</code> for the item at the given <code>index</code> .          |
| <code>removeItem(index)</code>        | Removes the item at the given <code>index</code> .                                 |
| <code>clear()</code>                  | Clears all items from the combo box.                                               |
| <code>setEditable(bool)</code>        | If <code>True</code> , the contents of the combo box are editable.                 |

Table **A-11** displays select signals for the combo box classes.

**Table A-11** Commonly used signals from the `QComboBox` and `QFontComboBox` classes

| Signal                                  | Description                                                                                  |
|-----------------------------------------|----------------------------------------------------------------------------------------------|
| <code>currentIndexChanged(index)</code> | Emitted if the current item in the combo box has changed.                                    |
| <code>currentTextChanged(text)</code>   | Signal emitted if the current item in the combo box has changed. Returns <code>text</code> . |
| <code>activated(index)</code>           | Emitted only if the user interacts with an item.                                             |
| <code>highlighted(index)</code>         | Emitted when an item in the combo box is highlighted.                                        |
| <code>textActivated(text)</code>        | Signal emitted when the user chooses an item.                                                |
| <code>currentFontChanged(font)</code>   | Emitted when the current <code>font</code> changes.                                          |

#### QLINEEDIT

The `QLineEdit` widget provides a single line for entering and editing plain text. Although not listed in the following tables, `QLineEdit` comes with `clear()`, `selectAll()`, `cut()`, `copy()`, `paste()`, `undo()`, and `redo()` slots already built-in. Table **A-12** displays a few the `QLineEdit` class's methods.

**Table A-12** Methods from the `QLineEdit` class

| Method              | Description                                  |
|---------------------|----------------------------------------------|
| <code>text()</code> | Retrieves the current text in the line edit. |

| Method                                                 | Description                                                                                                                                              |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>setAlignment (alignment)</code>                  | Sets the alignment of the text displayed in the widget.                                                                                                  |
| <code>setPlaceholderText (text)</code>                 | Displays placeholder text while line edit is empty.                                                                                                      |
| <code>setEchoMode (mode)</code>                        | <code>mode</code> describes how the contents of a line should be displayed. Set <code>mode</code> to <code>QLineEdit.Password</code> to mask characters. |
| <code>setMaxLength (int)</code>                        | Sets the maximum length of characters.                                                                                                                   |
| <code>setTextMargins (left, top, right, bottom)</code> | Sets the text margins for the text displayed in the line edit.                                                                                           |
| <code>setDragEnabled (bool)</code>                     | If <code>True</code> , dragging selected text in the line edit is permitted.                                                                             |

A few common signals for `QLineEdit` can be seen in Table [A-13](#).

**Table A-13** Commonly used signals from the `QLineEdit` class

| Signal                          | Description                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>returnPressed ()</code>   | Emitted when the Enter key is pressed. If a validator is set, then a signal is only emitted if the text is accepted. |
| <code>textChanged (text)</code> | Signal is emitted when the text changes.                                                                             |

#### TEXT EDITING WIDGETS

The two text editing classes, `QTextEdit` and `QPlainTextEdit`, provide tools and functionality for displaying and editing larger bodies of text. `QTextEdit` also has the added benefit of being able to work with rich text, graphics, and tables. Select methods for the two classes are displayed in Table [A-14](#). Both classes are similar to `QLineEdit`, because they already have editing features built-in.

Also worth noting is the `QTextBrowser` class, which inherits `QTextEdit`. `QTextBrowser` only allows read-only mode, but includes hypertext navigation functionality so that users can click links and follow them.

**Table A-14** Select methods from `QTextEdit` and `QPlainTextEdit`

| Method                          | Description                                         |
|---------------------------------|-----------------------------------------------------|
| <code>find (text, flags)</code> | Finds the next occurrence of text in the text edit. |
| <code>print (printer)</code>    | Print the text edit's document to the printer.      |

| Method                                | Description                                               |
|---------------------------------------|-----------------------------------------------------------|
| <code>setPlaceholderText(text)</code> | Sets <code>placeholderText</code> for text edit.          |
| <code>setReadOnly(bool)</code>        | If <code>True</code> , the text edit is set to read-only. |
| <code>toPlainText()</code>            | Returns the text of the text edit as plain text.          |
| <code>zoomIn(range)</code>            | Zooms in on the text.                                     |
| <code>zoomOut(range)</code>           | Zooms out on the text.                                    |

Commonly used signals for the text editing widgets can be found in Table [A-15](#).

**Table A-15** Select signals from `QTextEdit` and `QPlainTextEdit`

| Signal                          | Description                                                     |
|---------------------------------|-----------------------------------------------------------------|
| <code>selectionChanged()</code> | Signal emitted when the text selected in the text edit changes. |
| <code>textChanged()</code>      | Emitted whenever the contents of the text edit change.          |

#### SPIN BOX WIDGETS

Spin boxes allow users to choose values within a given range by clicking up/down buttons to cycle through the widget's values. Users can also manually type in values into the provided line edit. The `QAbstractSpinBox` class is the base class for the following classes:

1. `QSpinBox` – Handles integers.
2. `QDoubleSpinBox` – Similar to `QSpinBox` but is used for floating-point values.
3. `QDateTimeEdit` – A spin box widget for selecting dates and times. Use `setDisplayFormat()` to set the format used for displaying the dates and time.
4. `QDateEdit` – A spin box that displays only dates. Inherits from `QDateTimeEdit`.
5. `QTimeEdit` – A spin box that displays only times. Inherits from `QDateTimeEdit`.

Some of the methods for the `QSpinBox` and `QDoubleSpinBox` classes are listed in Table [A-16](#). The `QDateTimeEdit` and other spin box widgets have similar methods.

**Table A-16** Select signals from `QSpinBox` and `QDoubleSpinBox`. The value `val` refers to integers for `QSpinBox` and floating-point numbers for `QDoubleSpinBox`

| Method                     | Description                                      |
|----------------------------|--------------------------------------------------|
| <code>setValue(val)</code> | Sets the value <code>val</code> of the spin box. |

| Method                          | Description                                                                                          |
|---------------------------------|------------------------------------------------------------------------------------------------------|
| <code>setMinimum(val)</code>    | Sets the minimum value <code>val</code> of the spin box.                                             |
| <code>setMaximum(val)</code>    | Sets the maximum value <code>val</code> of the spin box.                                             |
| <code>setPrefix(str)</code>     | Adds a prefix to the start of the displayed value.                                                   |
| <code>setSuffix(str)</code>     | Adds a suffix to the end of the displayed value.                                                     |
| <code>setRange(min, max)</code> | Sets the minimum and maximum range values.                                                           |
| <code>setSingleStep(val)</code> | The spin box's value is incremented/decremented by <code>val</code> when the arrow keys are pressed. |

The `QSpinBox` and `QDoubleSpinBox` signals are found in Table [A-17](#).

**Table A-17** Signals from `QSpinBox` and `QDoubleSpinBox`

| Signal                         | Description                                                                         |
|--------------------------------|-------------------------------------------------------------------------------------|
| <code>valueChanged(val)</code> | Signal emitted when the value changes. Provides the new value's <code>val</code> .  |
| <code>textChanged(text)</code> | Signal emitted when the value changes. Provides the new value's <code>text</code> . |

#### SLIDER WIDGETS

The following widgets are different in appearance, but are actually quite similar in function. Widgets that inherit from the `QAbstractSlider` class are used for selecting integer values within a bounded range. Classes that inherit `QAbstractSlider` include the following:

1. `QDial` – Provides a rounded range controller for selecting or adjusting values. An example of `QDial` can be seen in Figure [A-1](#).
2. `QScrollBar` – Provides horizontal or vertical scrollbars that the user can use to access other parts of a document that are wider than the widget used to display it.
3. `QSlider` – Creates the classic horizontal and vertical slider widgets for controlling values within a specified range.

Table [A-18](#) shows some of the methods of the `QAbstractSlider` base class.

**Table A-18** Select methods from `QAbstractSlider`

| Method                       | Description                           |
|------------------------------|---------------------------------------|
| <code>value()</code>         | Holds the sliders current value.      |
| <code>setMinimum(int)</code> | Sets the minimum value of the slider. |



| Method                                   | Description                                                                                                                                   |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>setMaximum(int)</code>             | Sets the maximum value of the slider.                                                                                                         |
| <code>setOrientation(orientation)</code> | Sets the <code>orientation</code> to <code>Qt.Horizontal</code> or <code>Qt.Vertical</code> .                                                 |
| <code>setSingleStep(int)</code>          | The slider's value is incremented/decremented by <code>int</code> when the arrow keys are pressed.                                            |
| <code>setTracking(bool)</code>           | If <code>True</code> , the slider's position can be tracked.                                                                                  |
| <code>setSliderPosition(int)</code>      | Sets the current position of the slider.                                                                                                      |
| <code>setValue(int)</code>               | Sets the current position of the slider to <code>int</code> . If tracking is enabled, then this has the same effect as <code>value()</code> . |

Signals of the `QAbstractSlider` class can be found in Table [A-19](#).

**Table A-19** Signals from `QAbstractSlider`

| Signal                              | Description                                                                        |
|-------------------------------------|------------------------------------------------------------------------------------|
| <code>valueChanged(val)</code>      | Signal emitted when the value changes. Provides the new value's <code>val</code> . |
| <code>rangeChanged(min, max)</code> | Signal emitted when the range has changed with new minimum and maximum values.     |
| <code>sliderMoved(val)</code>       | Emitted when the slider is pressed down and the slider moves.                      |
| <code>sliderPressed()</code>        | Emitted when the slider is pressed down.                                           |
| <code>sliderReleased()</code>       | Emitted when the slider is released.                                               |

#### DISPLAY WIDGETS

The following widgets are all used for different purposes, but each has one major characteristic in common – they are all used for displaying information to the user.

#### QLABEL

The `QLabel` is a versatile widget. Although a label provides no user interaction functionality, `QLabel` is able to display plain or rich text, pixmaps, and even movies. Labels provide a number of methods for configuring their appearance. Table [A-20](#) lists a few of those methods.

**Table A-20** Select methods from `QLabel`

| Method                                | Description                                                  |
|---------------------------------------|--------------------------------------------------------------|
| <code>setPicture (picture)</code>     | Sets the label content to <code>picture</code> .             |
| <code>setPixmap (pixmap)</code>       | Sets the label content to <code>pixmap</code> .              |
| <code>setMovie (movie)</code>         | Sets the label content to <code>movie</code> .               |
| <code>setText (text)</code>           | Sets the label content to <code>text</code> .                |
| <code>setAlignment (alignment)</code> | Sets the alignment of the label's content.                   |
| <code>setIndent (int)</code>          | Sets the number of pixels that the label's text is indented. |
| <code>setMargin (int)</code>          | Sets the label's margins.                                    |

**QPROGRESSBAR**

Progress bars are used to give visual feedback to the user about the progress of a computer operation. Progress bars can be displayed vertically or horizontally. Table **A-21** shows some of the `QProgressBar` class's methods.

**Table A-21** Select methods for the `QProgressBar` class

| Method                                    | Description                                                                                  |
|-------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>value ()</code>                     | Holds the progress bar's current value.                                                      |
| <code>setMinimum (int)</code>             | Sets the progress bar's minimum value.                                                       |
| <code>setMaximum (int)</code>             | Sets the progress bar's maximum value.                                                       |
| <code>setRange (min, max)</code>          | Sets the minimum and maximum values.                                                         |
| <code>setOrientation (orientation)</code> | Sets the <code>orientation</code> , <code>Qt.Horizontal</code> or <code>Qt.Vertical</code> . |
| <code>setTextVisible (bool)</code>        | If <code>True</code> , the current completed percentage is displayed.                        |

`QProgressBar` has one signal , `valueChanged (int)` , that is emitted when the value shown in the progress bar changes.

**QGRAPHICVIEW**

The `QGraphicsView` class provides a widget for displaying the contents of a `QGraphicsScene`. As the one part of Qt's Graphics View Framework, the `QGraphicsView`'s responsibility is to display the items of a graphics scene in a scrollable window. The `QGraphicsScene`'s duty is to manage the items

in a scene. `QGraphicsItem` (or one of its subclasses) provides the items for a scene.

If you are interested in learning more about the Graphics View Framework, check out the following link:

<https://doc.qt.io/qt-5/graphicsview.html>

#### QLCDNUMBER

The **QLCDNumber** widget displays numbers in a seven-segment LCD display. An example of this can be seen in Figure A-1. The display can visualize decimal, hexadecimal, octal, and binary numbers. The LCD display can only display certain characters, so if an illegal character is passed, a space will be displayed in place of the character.



**Figure A-1** Example of the `QLCDNumber` and `QDial` widgets

Table A-22 lists a few of `QLCDNumber`'s methods.

**Table A-22** Select methods from the `QLCDNumber` class

| Method                                  | Description                                                                                                                           |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>value()</code>                    | Retrieves the LCD's displayed value.                                                                                                  |
| <code>intValue()</code>                 | Retrieves the displayed value rounded to the nearest integer value.                                                                   |
| <code>display(val)</code>               | Displays the value <code>val</code> in the <code>display</code> . <code>val</code> can be floating-point, integer, or string type.    |
| <code>setMode(mode)</code>              | Sets the <code>mode</code> of the LCD to display <code>Bin</code> , <code>Oct</code> , <code>Dec</code> , or <code>Hex</code> values. |
| <code>setSmallDecimalPoint(bool)</code> | If <code>True</code> , the decimal is drawn between two digits.                                                                       |

`QLCDNumber` has the`overflow()` signal, which is emitted when the widget is asked to display a number or string that is too long.

ITEM VIEWS

The following model view classes provide the means to display items in lists, tables, or tree structures. They must be used alongside a model class as part of Qt's model/view framework.

- 1. `QListView` – Provides a list and icon view for displaying items from a model
- 2. `QTableView` – Provides a table for displaying items from a model
- 3. `QTreeView` – Provides a hierarchical tree architecture for displaying items from a model

These classes all inherit from the`QAbstractItemView` class. Using signals and slots, item views created from `QAbstractItemView` are able to interact with models that use`QAbstractItemModel`. Each of the item views has their own methods for working with rows, columns, headers, and items. Views use the indices to manage items. You can find some of `QAbstractItemView`'s methods listed in Table [A-23](#).

PyQt also offers convenience item-based classes for each of the different types of views – `QListWidget`, `QTableWidget`, and `QTreeWidget`. Items are added to the widgets by using `QListWidgetItem`, `QTableWidget`, or `QTreeWidgetItem`.

Table A-23 Select methods for the `QAbstractItemView` base class

| Method                        | Description                        |
|-------------------------------|------------------------------------|
| <code>clearSelection()</code> | All items selected are deselected. |
| <code>selectAll()</code>      | Selects all the items in the view. |

| Method                                     | Description                                                             |
|--------------------------------------------|-------------------------------------------------------------------------|
| <code>setCurrentIndex(index)</code>        | Sets the item at <code>index</code> as the current item.                |
| <code>update(index)</code>                 | Updates the area at the given <code>index</code> .                      |
| <code>setAlternatingRowColors(bool)</code> | If <code>True</code> , the background is drawn with alternating colors. |
| <code>setAcceptDrops(bool)</code>          | If <code>True</code> , items can be dropped into the view.              |
| <code>setDragEnabled(bool)</code>          | If <code>True</code> , items can be dragged around in the view.         |
| <code>setIconSize(size)</code>             | Sets the <code>size</code> of icons.                                    |
| <code>setItemDelegate(delegate)</code>     | Sets an <code>itemdelegate</code> for the view's model/view framework.  |
| <code>setModel(model)</code>               | Sets the <code>model</code> for the view.                               |

Select signals for `QAbstractItemView` can be found in Table [A-24](#).

**Table A-24** Select methods for the `QAbstractItemView` base class

| Signal                            | Description                                                                                                 |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>activated(index)</code>     | Signal emitted when the item at <code>index</code> is activated by the user.                                |
| <code>clicked(index)</code>       | Emitted when the left mouse button is clicked on an item in the view (specified by <code>index</code> ).    |
| <code>doubleClicked(index)</code> | Emitted when a mouse button is double-clicked on an item in the view (specified by <code>index</code> ).    |
| <code>entered(index)</code>       | Signal emitted when the mouse cursor enters the item at <code>index</code> . Turn on mouse tracking to use. |
| <code>pressed(index)</code>       | Signal emitted when a mouse button is pressed on an item at <code>index</code> .                            |

#### CONTAINER WIDGETS

PyQt provides a few container widgets for maintaining control over groups of widgets. Containers can be used to manage input widgets and make the process of organizing a group of widgets simpler, or simply as a decorative widget for separating groups of widgets. Once a container is

created, a layout manager still needs to be applied to the container widget itself.

CONTAINERS WITH FRAMES

QFrame widgets can enclose and group widgets, as well as function as placeholders in windows. Using frames, you can set the appearance of other widgets to have raised, sunken, or flat appearances. The QFrame class is used as the base class for a few other container classes, including **QToolBox** and **QStackedWidget**. Table A-25 lists a few of QFrame’s methods.

Table A-25 Select methods for QFrame

| Method                               | Description                                                                                                          |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>setFrameRect (QRect)</code>    | Sets the rectangle that the frame is drawn in.                                                                       |
| <code>setFrameShadow (shadow)</code> | Sets the frame’s shadow, such as <code>Plain</code> , <code>Raised</code> , or <code>Sunken</code> .                 |
| <code>setFrameShape (shape)</code>   | Sets the frame’s shape, such as <code>Box</code> , <code>Panel</code> , <code>HLine</code> , or <code>VLine</code> . |
| <code>setLineWidth (int)</code>      | Sets the width of line drawn around the frame.                                                                       |

QToolBox widgets provide a series of pages or compartments in a column. To navigate through each of the pages, a tab is included at the top of each page. By clicking the next tab, the user can view a new tab’s contents.

Table A-26 A few of the QToolBox class’s methods

| Method                                        | Description                                                                           |
|-----------------------------------------------|---------------------------------------------------------------------------------------|
| <code>addItem (widget, text)</code>           | Adds the <code>widget</code> in a new tab at the bottom of the <code>toolbox</code> . |
| <code>insertItem (index, widget, text)</code> | Inserts the <code>widget</code> in a new tab at the given <code>index</code> .        |
| <code>indexOf (widget)</code>                 | Returns the index of the specified <code>widget</code> .                              |
| <code>setCurrentIndex (index)</code>          | Sets the <code>index</code> to a new item’s index.                                    |
| <code>setCurrentWidget (widget)</code>        | Makes the <code>widget</code> the current widget displayed in the toolbox.            |

When the item in a QToolBox is changed, the `currentChanged (index)` signal is emitted.

The QStackedWidget has a similar function to QToolBox, displaying multiple widgets stacked on top of one another to conserve space in a window. However, there is a key difference: QStackedWidget does not provide a means for the user to switch between tabs. Therefore, other widgets, such

as a `QComboBox` or a `QListWidget`, are used to navigate through the different pages.

The `QTabWidget` is another container class that is similar to `QStackedLayout`, but provides the tabs necessary to switch pages.

Finally, `QGroupBox` widgets typically group together collections of radio buttons and checkboxes. The main visual difference from the `QFrame` class is the addition of a title.

**QSCROLLAREA**

A scroll area can be added onto a child widget to display the contents within a frame. If the size of the frame changes, the scrollbars will appear, allowing the user to still view the entire child widget. The manner in how the scrollbars appear can be controlled with `QAbstractScrollArea`’s size policies. You can find a few of `QScrollArea`’s methods in Table [A-27](#).

**Table A-27** Select methods for `QScrollArea`

| Method                                             | Description                                                                                                                        |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>ensureVisible(x, y, xmargin, ymargin)</code> | Ensures the specified (x,y) coordinate with margins <code>xmargin</code> and <code>ymargin</code> remains visible in the viewport. |
| <code>setAlignment(alignment)</code>               | Sets the alignment of the scroll area’s widget.                                                                                    |
| <code>setWidget(widget)</code>                     | Sets the scroll area’s widget.                                                                                                     |
| <code>setWidgetResizable(bool)</code>              | If <code>False</code> , the scroll area abides by the child widget’s size.                                                         |

**QMdiAREA**

For multiple-windowed GUIs (MDIs), the `QMdiArea` provides the container for displaying multiple windows inside a single application window. Subwindows are instances of the `QMdiSubWindow` class and can be arranged in tiled or cascading patterns. The subwindows can work together, relaying information back and forth. A context menu could also be added to the MDI area widget as a means to conveniently switch between windows. Methods to help you get started using `QMdiArea` can be found in Table [A-28](#).

**Table A-28** List of select `QMdiArea` methods

| Method                            | Description                                     |
|-----------------------------------|-------------------------------------------------|
| <code>addSubWindow(widget)</code> | Adds widget as a new subwindow to the MDI area. |
| <code>activeSubWindow()</code>    | Returns the active subwindow.                   |
| <code>cascadeSubWindow()</code>   | Arranges subwindows in a cascade pattern.       |

| Method                                 | Description                                                                              |
|----------------------------------------|------------------------------------------------------------------------------------------|
| <code>tileSubWindows()</code>          | Arranges subwindows in a tiled pattern.                                                  |
| <code>removeSubWindow(widget)</code>   | Removes <code>widget</code> from the MDI area, where <code>widget</code> is a subwindow. |
| <code>setBackground(background)</code> | Sets the <code>QBrush</code> background for the MDI area.                                |
| <code>subWindowList(subwindows)</code> | Returns a list of <code>subwindows</code> .                                              |
| <code>setTabsClosable(bool)</code>     | If <code>True</code> , close buttons are placed on each tab in the tabbed view.          |
| <code>setTabsMovable()</code>          | If <code>True</code> , tabs within the tabbed view are movable.                          |

QT STYLE SHEETS

For a great reference to all of the widgets and properties that can be manipulated with Qt Style Sheets, have a look at the following link:

<https://doc.qt.io/qt-5/stylesheet-reference.html>

Style sheets allow for customizing many aspects and behaviors of widgets. Table A-29 lists many of the properties that can be modified. Widgets support only certain properties, so be sure to check out Qt’s documentation if you are not sure about which properties you can change.

Table A-29 List of properties that can be influenced using Qt Style Sheets

| Property                                | Description                                                                         |
|-----------------------------------------|-------------------------------------------------------------------------------------|
|                                         | The alternate background color for <code>QAbstractItemView</code> widgets.          |
| <code>alternate-background-color</code> | <pre>QListView{     alternate-background-color: blue;     background: grey; }</pre> |
| <code>background</code>                 | Shorthand for setting the background.                                               |
|                                         | Background color used for the widget.                                               |
| <code>background-color</code>           | <pre>QPushButton{     background-color: #49DE1F; }</pre>                            |



| Property                                             | Description                                                                                                                    |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| background-image                                     | <p>The background image used for the widget.</p> <pre>QFrame{ background-image: url(images/black_cat.png) }</pre>              |
| border                                               | <p>Shorthand for setting the widget's border.</p> <pre>QComboBox{ border: 2px solid magenta }</pre>                            |
| border-top, border-right, border-bottom, border-left | <p>Shorthand for specifying sides of the widget's border.</p>                                                                  |
| border-color                                         | <p>The color for all sides of the widget's border.</p>                                                                         |
| border-image                                         | <p>Specifies an image to fill the border.</p>                                                                                  |
| border-radius                                        | <p>The radius of the border's corners.</p> <pre>QTextEdit{ border-width: 1px; border-style: groove; border-radius: 3px }</pre> |
| border-style                                         | <p>Specifies the style for all of the border's edges.</p>                                                                      |
| border-width                                         | <p>Specifies the width for all of the border's edges.</p>                                                                      |
| color                                                | <p>The color used for rendering text.</p>                                                                                      |
| font                                                 | <p>Shorthand for defining a widget's font.</p> <pre>QRadioButton{ font: bold italic large "Helvetica" }</pre>                  |
| font-family, font-size, font-style, font-weight      | <p>Other properties used to individually set a font's features.</p>                                                            |
| height, width                                        | <p>The height and width of a widget .</p>                                                                                      |
| icon-size                                            | <p>The width and height of a widget's icon.</p>                                                                                |

| Property                                                                           | Description                                                                                                      |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>image</code>                                                                 | The image drawn on a widget. Can use <code>URL</code> or <code>SVG</code> .                                      |
| <code>left</code> , <code>right</code> , <code>top</code> ,<br><code>bottom</code> | Moves a widget by a certain offset relative to the parent's edge.                                                |
| <code>margin</code>                                                                | Specifies the widget's margins. Just like <code>border</code> , specific sides can also be set.                  |
| <code>max-height</code> , <code>max-width</code>                                   | The widget's maximum height or width.                                                                            |
| <code>min-height</code> , <code>min-width</code>                                   | The widget's minimum height or width.                                                                            |
| <code>outline</code>                                                               | The outline is used to draw a widget's border. Can also specify color, style, and radius.                        |
| <code>padding</code>                                                               | Specifies the widget's padding. Just like <code>border</code> , specific sides can also be set.                  |
| <code>selection-color</code>                                                       | The foreground color of selected items to text.                                                                  |
| <code>spacing</code>                                                               | Sets the internal spacing in a widget.                                                                           |
| <code>text-align</code>                                                            | Specifies the alignment of text and icons inside of a widget.<br><pre>QPushButton{     text-align: right }</pre> |

**SUMMARY**

Throughout this book, you have seen many of PyQt's foundational classes for building graphical user interfaces. Appendix A serves as a reference to help you analyze the programs found in this book and to learn more about the widgets, layouts, and style sheets used to design and build each application. The classes and methods contained here act as a guide to get you thinking about ways to improve on the programs in this guide and to help you to make your very own applications.

There is simply not enough room to include every class, method, or signal in this guide. As you follow along with the examples, use this appendix as a resource to help you learn and find out more about the possibilities of PyQt. If the answer isn't provided for you here, follow the links, search on the Internet, or send me an e-mail.

**APPENDIX B: PYTHON REFRESHER**

Python is a powerful and versatile open source programming language with a standard library packed with built-in modules, from providing access to system functionality such as file I/O to Internet data handling, to development tools, and much more. On top of the standard library, Python is also host to an extensive number of third-party modules which can be found in the Python Package Index (PyPI) repository. No matter what your project may be, you can almost be guaranteed to find a module to fit your needs. Of course, you could also create your own module and share it with the Python community.

Some of the uses for Python include

- Web and Internet development
- Database access
- GUI development
- Scientific and numeric applications
- Network programming
- Software and game development

The Python language also has many other noteworthy features, such as follows:

1. Python is an interpreted programming language. An **interpreted language** is one in which the code does not need to be compiled before it is run. The compiling step turns the code into machine language where the instructions can be directly executed by a computer's central processing unit (CPU). Since Python does not have a compilation step, the process to edit, test, and debug code is much faster.
2. Python is a language well-suited for creating dynamic applications. This is because Python is both **adynamically typed** language – type checking occurs only as the code is being run and variables can change types over time – and allows for **dynamic binding** – where methods that are called on objects are only checked during runtime.
3. While it is an object-oriented programming language, it does also support other programming paradigms such as procedural programming.

**Note** This section is not designed to act as a complete tutorial for Python. It focuses mainly on reviewing some of the basic data types, data structures, methods, and ideas that could be useful when creating your own GUI applications and reviewing other concepts that are used throughout this book.

First, let's get started by seeing if you have an updated version of Python on your computer. If not, take a moment to install it.

#### INSTALLING PYTHON

As of this writing, the current version of Python 3 is version 3.8, but the code in this book is also capable of being run on Python 3.6 or later.

Python is a cross-platform programming language and is therefore able to run on a number of different hardware platforms and operating systems.

Whether you are using Windows, MacOS, or some form of Linux, the simplest way to download the latest version of Python is to go to <https://python.org/downloads/> and find the installer for your platform.

#### GETTING PYTHON FOR WINDOWS

To first check if Python is installed on your system, open up a Command window and enter the command `python3` (all lowercase). If Python is already installed, you will get a notice telling you the current version that is on your system and a chevron, `>>>`, prompting you to enter a Python command. Otherwise, you will get an error message saying `python3` is not a recognized command.

If Python is not already on your computer, then downloading Python for Windows is fairly easy. Select the installer right for your system from [python.org](https://python.org/), download it, and follow along with the instructions in the dialog that appears. Be sure to include Python in your PATH by checking the box like in Figure B-1.

Once installation is complete, try entering the `python` command into the Command window again to make sure Python 3 is installed properly.



**Figure B-1** On Windows, make sure to check the box “Add Python 3.x to PATH”. This ensures that the interpreter will be placed in the correct directory

#### GETTING PYTHON FOR MACOS

Python already comes installed on MacOS, but will probably need to be updated. To check out which version of Python 3 you already have installed on your computer, open up the Terminal application and enter `python3`. Similar to Windows, if you get an error message, Python 3 is not installed on your system.

Doing some research online to find out what method works best for you is a good idea. Otherwise, you could end up with multiple versions of the Python interpreter on your computer.

Downloading Python directly and installing the interpreter from <https://python.org/> is one option. After the file downloads, simply run the installer. Once the installer is complete, check to make sure Python is installed by opening up the Terminal window and entering `python3` on the command line.

Another option is to download the Homebrew package manager. A package manager can be useful for maintaining, updating, and organizing the different software on your computer. If you are interested in using a package management system, first open a Terminal window on your Mac. To get Homebrew, go to <https://brew.sh> and copy and paste the code into Terminal to install it. Once the installation is complete, enter the following code (without the \$ symbol) into the terminal window to download Python using Homebrew:

```
$ brew install python3
```

Once the downloading process is complete, enter `python3` into the Terminal window to launch the Python interpreter.

If you have multiple versions of Python on your computer, you can also use Homebrew and a tool called `pyenv` to manage them.

GETTING PYTHON FOR LINUX

If you are using Linux , then you probably already have a version of Python installed on your system. The Linux platform has a number of different distributions – Ubuntu, Fedora, Debian, and a few others.

To check if Python is already installed and, if so, find out what the current version is, open the Terminal application on your system and enter `python3` . If Python 3 is installed, then your current version should be displayed along with the chevron prompt, `>>>` .

If you need to download or update your version of Python, you can download the most recent release from [python.org](https://python.org) . In Linux, it is also possible to download Python from the Terminal. However, the process for each distribution of Linux can be slightly different. There are a number of great tutorials online that you can follow along with if you choose to go this route to get Python installed on your computer.

DATA TYPES IN PYTHON

There are several data types already built into Python, including numeric, string, and Boolean types. The following section reviews what these data types look like, as well as takes a look at a few of Python’s built-in functions to work with them.

NUMERIC DATA TYPES

When working with numbers, Python has **integer** , **floating-point** , and **complex number** data types (Table **B-1** ).

Table B-1 Numeric data types in Python

| Numeric Data Type      | Representation             |
|------------------------|----------------------------|
| Integers               | 10, 254, 9876540           |
| Floating-Point Numbers | 5.7, 0.333, 0.5e6, 3.23e-5 |
| Complex Numbers        | 4+8j                       |

ARITHMETIC OPERATORS

Of course, Python also provides arithmetic operators and some built-in functions for performing mathematical operations (Table **B-2** ).

Table B-2 Arithmetic operators and expressions

| Operator                | Example                          |
|-------------------------|----------------------------------|
| Addition/Subtraction    | <code>x + 5, 20 - j</code>       |
| Multiplication/Division | <code>n * 10, 5 / m (m≠0)</code> |
| Exponents               | <code>n ** 3</code>              |
| Modulo Operation        | <code>100 % d</code>             |

WORKING WITH NUMERIC DATA TYPES

Table B-3 lists a few of the built-in mathematical functions in Python.

Table B-3 Python methods for working with numeric values

| Functions                  | Description                                                      |
|----------------------------|------------------------------------------------------------------|
| <code>abs(value)</code>    | Returns the absolute value of a number.                          |
| <code>max(iterable)</code> | Returns the largest value in a list or other iterable object.    |
| <code>min(iterable)</code> | Returns the smallest value in a list or other iterable object.   |
| <code>round(value)</code>  | Rounds a floating-point number and returns an int value.         |
| <code>sum(iterable)</code> | Returns the sum of the items in a list or other iterable object. |

STRING DATA TYPE

Sequences of textual data can be represented using **strings**. Strings can be delimited with single or double quotes (Table B-4).

Table B-4 String data type examples in Python

| Numeric Data Type           | Representation                                                                         |
|-----------------------------|----------------------------------------------------------------------------------------|
| String Examples             | <code>'This is a string literal.'</code><br><code>"String with numbers - 1234."</code> |
| Empty String                | <code>''</code> , <code>""</code>                                                      |
| Escape Sequences in Strings | <code>"Include quote \" character."</code>                                             |
| Raw Strings                 | <code>r"Escape \nchars \t ignored."</code>                                             |

The escape character backslash, ‘\’, can be used to include quotes or whitespace characters, such as ‘\n’ for a newline or ‘\t’ for a tab, in a string.

Including ‘r’ before the beginning of a string completely ignores all escape characters. Raw strings are very useful for regular expressions. Triple quotations can also be used to create multiline strings.

WORKINGS WITH STRINGS

Strings in Python are **immutable** – these are objects that cannot be modified after they are created. Whenever you apply an operation to a string, a copy is created. Parts of a string, or substring, can also be accessed using indices and slices. The following bit of code looks at creating and accessing parts of strings:

```
a_string = "I like dogs." # Variable assignment
print(a_string[4]) # Select and print item at index 4
print(a_string[7:10]) # Print items from index 7 to index 9
```

Individual characters in a list can be accessed by selecting the index value associated with that character. The first item in a string has an index value of 0. Using `a_string[4]`, the letter ‘k’ is selected. You can also start from the end of the string and work backward by using negative index values, starting from -1.

Use slicing to select a longer substring. In the preceding example, `a_string[7:10]` will include everything from `a_string[7]` to `a_string[9]`, leaving out the final character in the string at index 10.

To test if a substring is or is not contained within a larger string, use the special membership operators, `in` and `not in`.

```
a_string = "Connor, Sarah"
print('Sarah' in a_string) # Returns and prints True
print('John' not in a_string) # Returns and prints True
```

It is also possible to manipulate the content of strings. Concatenation is the process of combining strings and can be achieved using the addition symbol, `+`.

```
first_name = "Peter"
last_name = "Parker"
full_name = first_name + " " + last_name # 'Peter Parker'
```

Finally, Table **B-5** lists some of the other built-in methods for working with strings. Methods are similar to functions, except they are “called on” an object using specific keywords and a period, ‘.’. Methods such as `isupper()` and `islower()` are useful for testing and comparing strings and validating user input.

Table B-5 More methods of the String class

| String Methods       | Description                                      |
|----------------------|--------------------------------------------------|
| <code>upper()</code> | Converts all letters in the string to uppercase. |
| <code>lower()</code> | Converts all letters in the string to lowercase. |

| String Methods                           | Description                                                                                                                                     |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>isupper()</code>                   | Returns <code>True</code> if the string has at least one letter and all letters are uppercase.                                                  |
| <code>islower()</code>                   | Returns <code>True</code> if the string has at least one letter and all letters are lowercase.                                                  |
| <code>isalpha()</code>                   | Returns <code>True</code> if the string contains only letters and is not blank.                                                                 |
| <code>isalnum()</code>                   | Returns <code>True</code> if the string contains only letters and numbers and is not blank.                                                     |
| <code>isdecimal()</code>                 | Returns <code>True</code> if the string contains only numeric characters and is not blank.                                                      |
| <code>startswith(str, begin, end)</code> | Returns <code>True</code> if the string or sub-string value begins with specified prefix.                                                       |
| <code>endswith(str, begin, end)</code>   | Returns <code>True</code> if the string or sub-string value ends with specified prefix.                                                         |
| <code>count(str, begin, end)</code>      | Returns the number of times a specified string occurs in a string or substring.                                                                 |
| <code>delimiter.join(iterable)</code>    | Concatenates a list of strings together into a single string value. Use the delimiter to specify how the individual strings are separated.      |
| <code>split(delimiter, maxsplit)</code>  | Takes a single string object and returns a list of strings. Use the <code>delimiter</code> to specify where in the string a split should occur. |
| <code>strip(chars)</code>                | Removes specified <code>chars</code> and white-spaces from the beginning and the ending in a string.                                            |

**BOOLEAN DATA TYPE**

Objects in Python 3 can be assigned one of two Boolean values, `True` or `False`. Non-Boolean objects, ones that are assigned integer or string values, can also be evaluated using Boolean operators and expressions. This comes in handy in



situations where you need to check if two strings match or to compare two integer values.

```
num_1, num_2 = 35, 10 # Ex of multiple assignment statement
print(num_1 == num_2) # Prints False
print(num_1 > num_2) # Prints True
```

DATA STRUCTURES IN PYTHON

As you begin working with more data, you will need a way to organize, store, and access that information. Python includes a few data structures with their own special sets of rules – lists, tuples, sets, and dictionaries.

LISTS

A **list** is a mutable data structure that is created using brackets:

```
# List with mixed data types
a_list = ["John Doe", 23, 1976, "blue"]
# Example of nested list
nested_list = ["zebra", "tiger", "turtle", [3, 2, 7]]
```

**Mutable** objects are ones whose content can be modified without creating a new object. You can append, remove, or even rearrange items within the list. Similar to strings, items in lists are in an ordered sequence and can also be accessed using indices and slicing.

```
print(a_list[2]) # Access and print item at index 2
print(a_list[0:3]) # Only print items from indices 0 to 2
print(a_list[:]) # Print the entire list
```

To access the values inside nested lists

```
print(nested_list[0]) # Prints ['zebra', 'tiger', 'turtle']
print(nested_list[1][1]) # Prints 2
```

To get the length of a list or other iterative objects such as strings or tuples, use the built-in function `len()` :

```
print(len(a_list)) # Prints 4
```

Similar to strings, lists and other iterables can also use the `in` and `not in` operators to check if values are present in the list.

```
print("John Doe" in a_list) # Prints True
```

Table **B-6** contains some useful methods for working with lists.

**Table B-6** Some List class methods

| List Methods               | Description                                                        |
|----------------------------|--------------------------------------------------------------------|
| <code>append(value)</code> | Appends a single item to the end of a list.                        |
| <code>count(value)</code>  | Returns the number of times a <code>value</code> occurs in a list. |

| List Methods                       | Description                                                                 |
|------------------------------------|-----------------------------------------------------------------------------|
| <code>index (value)</code>         | Returns the index of the first occurrence of <code>avalue</code> in a list. |
| <code>insert (index, value)</code> | Inserts <code>onevalue</code> at a <code>specificindex</code> location.     |
| <code>extend (iterable)</code>     | Adds more than one item to the end of a list.                               |
| <code>pop (index)</code>           | Removes the item at the <code>specifiedindex</code> .                       |
| <code>remove (value)</code>        | Removes the first occurrence of <code>avalue</code> in a list.              |
| <code>clear ()</code>              | Clears the items in a list.                                                 |

`sort ()` Sorts the items in a list.

It is also possible to remove items or an entire list using the `del` statement.

```
del a_list[3] # Delete item at index 3
# Delete multiple items using slicing
del a_list[0:2]
del a_list # Delete the entire list
```

TUPLES

A **tuple** is an immutable data structure that is created using parentheses:

```
# Tuple with mixed data types
a_tuple = ("Jane Doe", 25, 1982, "brown")
```

It is possible to find values in a tuple using indices and slicing just like lists. However, since tuples are immutable, actions such as appending, sorting, or replacing items cannot be performed on them. This can be very useful when you have data that you may want others to view, but not have the ability to alter.

Many of Python’s built-in functions such as `max ()` and `len ()` can also be used with tuple objects (Table **B-7** ).

**Table B-7** Tuple class methods

| Tuple Methods              | Description                                                                  |
|----------------------------|------------------------------------------------------------------------------|
| <code>index (value)</code> | Returns the index of the first occurrence of <code>avalue</code> in a tuple. |
| <code>count (value)</code> | Returns the number of times <code>avalue</code> occurs in a tuple.           |

SETS

**As**et is a mutable data structure that is created using curly brackets:

```
# Set with mixed data types
a_set = {"John Smith", 45, 2001, "brown"}
```

Generally, sets are thought of as a collection of data of the same type, but they can also contain different data types. Sets in Python are unordered and unindexed and do not allow for duplicate values. Therefore, methods such as `asindex()` cannot be used with them. Sets can be very useful for removing repeated values and for performing mathematical operations from set theory on data. A few of those methods can be found in Table [B-8](#).

**Table B-8** A few important Set class methods

| Set Methods                    | Description                                                |
|--------------------------------|------------------------------------------------------------|
| <code>add(value)</code>        | Adds a single item to a set.                               |
| <code>update(iterable)</code>  | Adds multiple items to a set.                              |
| <code>remove(value)</code>     | Removes the specified <code>value</code> from a set.       |
| <code>clear()</code>           | Empties all items in the set.                              |
| <code>difference(set)</code>   | Returns the difference of two or more sets as a new set.   |
| <code>intersection(set)</code> | Returns the intersection of two or more sets as a new set. |
| <code>union(set)</code>        | Returns a set that contains the union of two or more sets. |

**DICTIONARIES**

**Ad**ictionary is a mutable data structure composed of key/value pairs that is created using curly brackets:

```
# Example of dictionary
a_dict = {"name": "Jane Smith", "age": 29, "year": 1970, "eye color": "green"}
```

In the preceding example, “name” is a **key** and “Jane Smith” is **value** associated with it. The key is similar to the index in lists. You can use the key to access and organize specific items in the dictionary.

Even though dictionaries are unordered, it is very easy to access their contents using the keys. To find out a key’s value, look at the following bit of code:

```
print(a_dict["age"]) # Prints value associated with "age"
```

There are also a few methods that can help you to work with the items in dictionaries:

- `keys()` – Returns a list of all of the dictionary’s keys
- `values()` – Returns a list of all of the dictionary’s values

- `items()` – Returns a list containing a tuple for each key/pair value

For example, the following bit of code will return a list of all of the values

in `a_dict`:

```
dict_values = list(a_dict.values()) # Create a list using typecasting
print(dict_values) # ['Jane Smith', 29, 1970, 'green']
```

It is also possible to check if keys or values exist in a dictionary using the membership operators `in` and `not in`. By default, if you do not specify keys or values, Python will search through the keys. The following example is equivalent to searching for `"year"` in `a_dict.keys()`:

```
print("year" in a_dict) # Prints True
```

To add a new key/value pair to the dictionary, you could use an `if` statement to first check if the key already exists or not. If it does not, then add the new item to the dictionary.

```
if "height" not in a_dict:
    a_dict["height"] = 1.82
```

Another way to check if an item can be found in a dictionary is to use the `setdefault()` method. If the key does not already exist or does not have a value, then the key/value pair is added. However, if the key already exists and has a value, then the `setdefault()` method does not make any changes to the dictionary. The following code will add the `"hair"` key and its value to `a_dict`:

```
a_dict.setdefault("hair", "brown")
```

Table [B-9](#) lists additional Dictionary class methods.

**Table B-9** Some other Dictionary class methods

| Dictionary Methods                   | Description                                                                                                                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>get(key, default value)</code> | Returns the value of the specified <code>key</code> . If the key does not exist, then <code>get()</code> uses the <code>default value</code> . Useful for avoiding errors. |
| <code>copy()</code>                  | Creates a copy of a dictionary.                                                                                                                                            |
| <code>update({key: value})</code>    | Adds a new key/value pair to a dictionary.                                                                                                                                 |
| <code>pop(key)</code>                | Removes an item from the dictionary by specifying the <code>key</code> .                                                                                                   |
| <code>clear()</code>                 | Clears all of the items from a dictionary.                                                                                                                                 |

**DATA TYPE CONVERSION**

**Typecasting** is the action of directly converting one data type to another. In Python, it is possible to convert integers to floating-point numbers, integers to

strings, lists to tuples, as well as other types of conversions (Table [B-10](#)). The following example shows how to convert a list into a dictionary:

```
info = [{"name", "Sam"}, {"age", 12}]
print(dict(info)) # {'name': 'Sam', 'age': 12}
```

Table B-10 Typecasting functions

| Types           | Typecasting Functions          | Description                                                                                         |
|-----------------|--------------------------------|-----------------------------------------------------------------------------------------------------|
| Arithmetic      | <code>int (value, base)</code> | Converts different data types to int values. Specify the <code>base</code> if converting a string.  |
|                 | <code>float (value)</code>     | Converts integer to floating-point value.                                                           |
|                 | <code>ord (character)</code>   | Converts a single character to an integer.                                                          |
| Strings         | <code>str (value)</code>       | Converts other data types into a string.                                                            |
| Data Structures | <code>list (iterable)</code>   | Converts an iterable object into a list.                                                            |
|                 | <code>tuple (iterable)</code>  | Converts an iterable object into a tuple.                                                           |
|                 | <code>set (iterable)</code>    | Converts an iterable object into a set. May result in some data loss if there are duplicate values. |
|                 | <code>dict (iterable)</code>   | Converts an iterable with structure (key, value) into a dictionary.                                 |

CONDITIONALS AND LOOPS IN PYTHON

The following section takes a look at programming tools that are used for controlling the flow and execution of commands and repetitive tasks while a program is running. Any of the following statements can be placed inside of another to create **nested loops**.

“IF-ELIF-ELSE” CONDITIONAL STATEMENTS

**Conditional statements** are used in programming to decide whether or not to perform certain actions based on whether the specified Boolean constraints evaluate to `True` or `False`. These types of statements are handled in Python using `if-elif-else` statements.

The following code shows a simple example of how to use `if` statements to check if items exist in a list:

```
car_types = ["economy", "sedan", "convertible", "SUV", "economy"]
if "SUV" not in car_types:
    car_types.append("SUV")
elif car_types.count("economy") > 1:
    car_types.remove("economy")
else:
    car_types.append("luxury")
print(car_types) # ['sedan', 'convertible', 'SUV', 'economy']
```

The first `if` statement evaluates to `False` since "SUV" already exists in the `car_types` list. The `elif` statement provides other conditions to check if previous clauses were not true. Finally, `else` statements are executed if all previous conditions were false.

There are other operators besides `in` and `not in` that can be used for evaluating Boolean expressions. Table [B-11](#) lists some of them.

Table B-11 Python operators

| Types      | Operators                           | Description                                                                            |
|------------|-------------------------------------|----------------------------------------------------------------------------------------|
| Comparison | <code>x == y</code>                 | Equal to                                                                               |
|            | <code>x != y</code>                 | Not equal to                                                                           |
|            | <code>x &gt; y</code>               | Greater than                                                                           |
|            | <code>x &lt; y</code>               | Less than                                                                              |
|            | <code>x &gt;= y</code>              | Greater than or equal to                                                               |
| Logical    | <code>x &gt; 5 and y &lt; -5</code> | Returns <code>True</code> if both statements are true.                                 |
|            | <code>x &gt; 5 or y &lt; -5</code>  | Returns <code>True</code> if both statements or only one of them is true.              |
|            | <code>not (x &gt; 5)</code>         | Returns the opposite . Changes <code>True</code> to <code>False</code> and vice versa. |

“FOR” LOOPS

Python is very useful for automating repetitive tasks using loops. The `for` loop is very useful when you have a task that needs to be executed a certain number of times, such as traversing through a sequence until you reach the end.

```
colors = ["blue", "red", "purple", "green", "white"]
for color in colors:
    print("Current color: {}".format(color))
```

The preceding code will cycle through the list five times, once for each item in the list. There are also ways to repeat a block of code or iterate over a sequence of numbers using the `range()` function. The following code will print out the index values and their corresponding colors from the `colors` list:

```
for i in range(len(colors)):
    print("{} . {}".format(i + 1, colors[i]))
```

The `enumerate()` function is also a way to include a counter while iterating through a sequence.

```
for count, color in enumerate(colors, start=1):
    print("{} . {}".format(count, color))
```

#### LIST COMPREHENSIONS

Now that we have gone over `for` loops and `if` statements, let's take a look at a very important concept in Python, **list comprehensions**. List comprehensions are a way to create new lists from more compact code.

Given a list of strings and integers, the following example uses list comprehension to create a new list of only the string values:

```
a_list = ["Sam", 1978, "Elsa", 1984, "Marcus", 1980, "Trevor", 1983]
new_list = [word for word in a_list if type(word) == str]
print(new_list) # ['Sam', 'Elsa', 'Marcus', 'Trevor']
```

Using nested loops, the following code is equivalent to the preceding example:

```
new_list = [] # Creates an empty list
for word in a_list:
    if type(word) == str:
        new_list.append(word)
print(new_list) # ['Sam', 'Elsa', 'Marcus', 'Trevor']
```

#### "WHILE" LOOPS

Unlike the `for` loop, the `while` loop can be used to execute a block of code an unknown number of times, as long as the Boolean condition being tested continues to evaluate to `True`. When the computer reaches the end of a `while` clause, it returns back to the beginning of the loop and checks again if the condition is still true. If so, the clause is executed once more, and the condition will be checked again at the end.

```
while True:
    print("Please enter your age: ")
    age = int(input())
    if age < 21:
        print("You are underage.")
        continue
    else:
        break
print("Access granted.")
```

For this example, the `while` condition will always evaluate to `True`. However, you could also use other logical or comparison operators to test different conditions. Once in the loop, the user is asked to input their age. This information is then handled using an `if-else` conditional statement.

In addition, there are types of statements that can be used to control the flow of the program and handle exceptions in a `while` loop - `break` and `continue`. These statements can also be used in `for` loops.

- `break` – Used to immediately exit a `while` loop. After exiting, the program will continue with the statement following the loop.
- `continue` – Used to immediately skip the rest of the loop and return back to start of the `while` loop. The condition statement is then reevaluated with the next iterative value.

#### FUNCTIONS

A **function** is a collection of statements that perform some particular task and can be reused multiple times throughout a program. Functions also make code easier to read by avoiding duplicate code.

You can also define your own functions in Python using the `def()` statement. When you call a function by invoking its name, you can also pass values, known as arguments, between the parentheses. Since functions in Python are treated as objects, you can also pass them as arguments to other functions. They also always return values whether it is `None` or some calculated value. Functions can even return other functions.

```
def check_for_nums(items_list):
    """
    Checks to see if string (item) contains only
    letters. If the string contains a numeric value,
    it is removed from the list.
    """
    for item in items_list[:]:
        if item.isalpha() != True:
            items_list.remove(item)
    return items_list # Return statement and value
test_list = ["horse", "1234", "d0g", "mouse", "m3"]
new_list = check_for_nums(test_list) # Function call
print(new_list) # Prints ['horse', 'mouse']
```

The function `check_for_nums()` is used to iterate through a list and remove any strings that contain numeric values. The function takes as an argument a list of string values. When `items_list` is called in the `for` loop, we need to iterate over a copy of the list (created using `[:]`) rather than the actual list itself in order to modify it using `remove()`. The `for` loop could also be written more concisely using list comprehension.

Be sure to consider local and global scope when creating functions and passing arguments between them. Variables and values created in a called function only exist within that function's **local scope**. Other variables and parameters created outside of all functions exist in the **global scope** and can be accessed by all functions.

#### LAMBDA FUNCTIONS

**Lambda functions** are often referred to as anonymous functions, or functions that have not been assigned a name. They are single expressions that can take multiple arguments. The general form for a lambda function is

```
lambda arguments : expression
```

The following example demonstrates how to create and call a multiargument lambda function in Python:



```
full_name = lambda f, l: "Full name: {} {}".format(f, l)
print(full_name("Ben", "Franklin"))
```

This is equivalent to creating a function using `def()`.

```
def full_name(first, last):
    print("Full name: {} {}".format(first, last))
```

Lambda functions are very useful when creating GUIs for mapping actions in response to events. When a button is clicked and a signal is triggered, a method, also known as a slot, handles that event. A method could be the call to a function, but in some instances, a lambda function can be used for the same reason.

#### OBJECT-ORIENTED PROGRAMMING (OOP)

In this section we will take a brief look at what object-oriented programming is and how to create objects using classes.

There are a number of different **programming paradigms**, or methods and styles of programming. One common method is **procedural programming** where a computer follows a sequential set of commands to perform some task. This kind of programming can make it difficult for adding new functions and working with more dynamic situations.

Another approach is **object-oriented programming (OOP)** which focuses on creating objects with their own properties and behaviors, and modeling their relationships between other objects. Objects are created using classes – which act as templates for the data, attributes, and methods that can be applied to an instance of a class. An **instance**, or an object created from a class, has access to all the data and methods inside the class.

OOP also introduces the idea of **inheritance**, which is the concept of creating new classes that derive properties and behaviors from existing classes. An object created using inheritance is known as a **child**. A child object has its own set of attributes and also acquires all of the properties and behaviors from its **parent** class.

The following example demonstrates how to create a new class, `Window`, for GUI development which inherits from the PyQt5 class, `QMainWindow`:

```
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow
class Window(QMainWindow):
    """
    Create a new class, Window, which inherits from other PyQt modules, in this case from the QMainWindow
    """
    def __init__(self):
        super().__init__() # super is used to access methods from parent class
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(100, 100, 300, 200)
```

```

        self.setWindowTitle("Create the Window Class")
        self.show()
    app = QApplication(sys.argv) # Create instance of QApplication class
    window = Window() # Create instance of Window class
    sys.exit(app.exec_()) # Start the event loop

```

The `Window` object is an instance of the `Window` class. To make code easier to understand, classes start with an uppercase letter. The child object, `window`, inherits from both `Window` and from the `QMainWindow` class and is able to use the methods found in `QMainWindow`, such as `setGeometry()` and `show()`, to create our GUI window.

In order to reference the current class, the keyword `self` allows us to use any of the data or methods within `QMainWindow` as well as any new methods we create in the `Window` class.

#### EXCEPTION HANDLING IN PYTHON

It is quite common for programs to run into errors when they are running. **Syntax errors** are the ones you will most often run into when writing your code. They are caused by incorrect syntax such as forgetting to include a colon, `:`, after a `for` clause.

**Exceptions** are the errors that occur when executing code. It is important to consider how your program will handle these exceptions rather than allowing your application to crash. Python includes a number of built-in exceptions, including how to handle not being able to locate or open a file, incorrect data types being passed into a function, dividing by zero, and various other situations.

The best way to handle an exception is to use the `try` and `except` statements. The following code shows how to check if a user correctly enters an integer value for their age:

```

while True:
    try:
        weight = int(input("Enter your weight (in lbs): "))
        break
    except ValueError:
        print("Invalid input.")

```

Code that could possibly cause an error is placed in the `try` clause. If an exception does occur while the program is running, then the `except` clause will catch it and handle the error accordingly. Depending upon the situation, you could display an error message, use the `break` or `continue` statements, or force a specific kind of error to occur.

#### READING AND WRITING TO FILES IN PYTHON

Python also provides built-in modules and functions to open, append, and write to files. Working with and locating files in directories can be a very long topic that includes filename pattern matching, traversing directories, deleting files and directories, and more. This section will simply focus on the basics of opening and writing to files.

The simplest way to open a file is to use

```
with('path to file', mode) as f
```

There are other ways to open files, but using this pattern allows for cleaner code and ensures that files are attended to by closing the file once the function using the resource is finished executing.

```
with open ('quotes.txt', 'w') as f:
    text = "Life is what happens when you're busy making other plans. -John Lennon"
    f.write(text)
```

The `open()` method takes as arguments the path to the file and a mode to tell the computer how to handle the file. The second argument, `'w'`, means we want to write to a file and ensures that a new file is created if it does not exist already. The `text` is then written to a new file using `f.write()`. Other options to interact with files include reading the contents of an entire file with `f.read()` or only reading one line at a time using `f.readline()`.

Table [B-12](#) lists the common modes for reading and writing to files.

**Table B-12** Some modes for working with files

| Character | Definition                                                                                                                          |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| 'r'       | Opens a file for reading. This is the default mode.                                                                                 |
| 'w'       | Opens a file for writing. Overwrites a file if one already exists. Otherwise, creates a new file.                                   |
| 'a'       | Opens a new file for appending. File pointer starts at the end of the file if it exists. Otherwise, creates a new file.             |
| 'r+'      | Opens a file for reading and writing.                                                                                               |
| 'w+'      | Opens a file for writing and reading. Overwrites a file if one already exists. Otherwise, creates a new file.                       |
| 'a+'      | Opens a new file for appending and reading. File pointer starts at the end of the file if it exists. Otherwise, creates a new file. |

**SUMMARY**

Hopefully this appendix helps you to recall some method you may have forgotten about, or sparks an idea in your mind about how to solve a problem when creating your own GUIs, or maybe even helps you to learn something new about programming in Python.

This appendix covers information from data types and data structures to conditional statements and iteration using loops, to creating your own functions and classes, and more. But it only skims the surface of the possibilities of what you could apply in your own applications.

There are so many possibilities for working with PyQt and Python, and covering all of that information could definitely be written in more than

one book. If you ever get stuck, the Internet is definitely an amazing resource full of information and guidance to working through your problems.

Best of luck in all of your projects and in your endeavors!

---

---

## Index

### A

- `about()` method
- Account management GUI
  - editing data
  - input/update/delete
  - project parts
- QStringQuery
- QStringRelationalDelegate
- QStringRelationalTableModel class
- QStringTableModel class
- table accounts
- Action Editor dock widget
- `addAction()` method
- `addBindValue()` method
- `addButton()` method
- `addDatabase()` function
- `addItem()` method
- `addLayout()` method
- `addMenu()` method
- `addStretch()` method
- `addWidget()` / `addLayout()` methods
- `addWidget()` method

### B

- `backgroundTab()` method
- Binding
- Boolean data type
- `buttonClicked()` function
- Button widgets
- ButtonWindow class

### C

- `calculateTotal()` method
- Calendar GUI
- code
  - importing modules
- Camera GUI
- MDI applications
  - camera's viewfinder
- Cascading Style Sheets (CSS)
- `cb_text` widget
- Central processing unit (CPU)
- `central_widget`
- `changeHeader()` method
- Classes
  - dialog boxes
  - event handling
  - layout managers
- QApplication
- QWidget
  - `clearEntries()` function
  - `clicked()` signal
  - `clickLogin()` method

**Clipboard**`clipboard_dock` widget`closeEvent()` method`close()` function**Conditional statements****if-elif-else****for loops****while loops**`confirmSignUp()` function**Context menus/pull-down menus**`copyFromClipboard()` method`createClipboard()` method`createConnection()` method`createMenu()` method`createNewUser()` method`createNotepadWidget()` method`createTable()` method`cubicTo()` method`currentChanged()` signal`currentColumn()` method`currentDate()` method`currentDateTime()` method`current_date_edit` widget`currentTime()` method**Custom signals****D**`dataChanged()` method`dateChanged()` signal**Data structures****dictionaries****list****sets****tuples**`def()` statement**Dialog boxes****Directory viewer GUI****code****hierarchical file system****menu****model/view paradigm**`displayButton` function`displayCheckBoxes()` method`displayLabels()` function`displayMessageBox()` function**Display widgets****QGraphicsView****QLabel****QLCDNumber****QProgressBar**`drawBackground()` method`drawCurves()` method`drawLine()` method`drawOnCanvas()` method`drawPoint()` method`drawRect()` method`drawText()` method**Dynamic binding****E****EntryWindow** class

Event handling

## **F**

Food ordering GUI

applying style sheets

code

design

profile details tab

QGroupBox widget

QRadioButton

QTabWidget class

types

formWidgets() method

## **G**

getDouble() method

getInt() method

getItem() method

getMultiLineText() method

getOpenFileName() method

getPixelValues() method

getText() method

Gradients

Graphical user interfaces (GUIs)

Graphics View framework

GUI, drag/drop

GUI, sticky notes

## **H**

Hangman GUI

code

DrawingLabel class

newGame() method

hasText() method

## **I, J**

ImageDemo class

initializeUI() function

\_\_init\_\_() method

Input widgets

QComboBox class

QLineEdit

Interface design

Interpreted language

isChecked() method

isValid() method

Item views

## **K**

Keypad GUI

apply layouts, Qt Designer

create/edit Python code

Edit Signals/Slots mode

frame objects

grid layout

keypad.ui

New Form dialog box

properties

Property Editor

QFrame class

QLineEdit widgets

QPushButton widgets

retranslateUi() method

setupUi() method

style sheet creation

- vertical layout
- `keyPressEvent()` function
- L**
- Lambda functions
- Layout management
- absolute positioning, `move()`
- notepad GUI
- `QFileDialog` class
- `QTextEdit` widget
  - definition
  - methods
- Nesting
- Notepad GUI, solution
- code
- `QTextEdit` widget
- `QHBoxLayout/QFormLayout` classes
- `QTextEdit` widget
- Widgets/classes
- Layout manager
- List comprehensions
- `loadCSVLAYOUT()` method
- `loadProgress()` function
- Login GUI
- code
- key components
- layout
- `QLineEdit` widget
- `LoginUI` class
- M**
- `menuBar()` function
- Menus
  - definition
  - pull-down
- `QAction` class
- `QIcon` class
- application icon
- `icon_button`
- pixmaps
- `QPushButtons`, code
- settings
- `QMainWindow` classvs. `QWidget`
- `QMenuBar` class
- structure
- submenus, checkable items
- using PyQt
- Model-view-controller (MVC)
- Model/view programming
- components
- CSV file
- PyQt
- table creation
- `mouseMoveEvent()` method
- `mousePressEvent()` method
- `move()` method
- `moveMouseEvent()` method
- Multiple-document interface (MDI)
- Multipurpose Internet Mail Extensions (MIME)
- Multithreading
- N**
- `name_entry` widget

- Nesting layouts
- New user GUI
  - code
  - creation
  - notepadMenu() method
  - numberClicked() slot
- Numeric data types
- O**
  - Object Inspector dock widget
- Object-oriented programming (OOP)
- openImage() method
- P**
  - Painter GUI
    - Canvas class
    - creation
    - mouse movement
    - tool tips
    - variables/objects
  - PainterWindow class
  - paintEvent() function
  - paste\_button widget
  - paste() method
  - pasteText() method
- Photo editor GUI
  - pixmap property
  - pizzaTab() method
  - prepare() method
  - printImage() method
  - printToTerminal() function
- Pseudostates
- PyQt
  - About dialog
  - classes
  - events
  - exec\_()
  - framework
  - definition
  - Qt designer
  - requirements
  - signal/slot mechanism
  - source code
  - Tkinter
  - uses
  - QColorDialog
  - QFontDialog
  - QInputDialog
- PyQt5
  - Anaconda distribution
- Classes
  - See* Classes
- Linux(Ubuntu)
- MacOS
- modules
- windows
- Python
  - exception handling
- features
- Linux
- MacOS
- opening/writing, files



uses  
windows  
Python Package Index (PyPI)  
pyuic5 utility  
**Q**  
QAnimationProperty class  
QApplication  
QApplication style sheet,  
QBrush class  
QCalendarWidget class  
QCamera class  
QCheckBox widget  
QClipboard class  
QColor class  
QDateEdit widgets  
QDockWidget class  
QFileSystemModel class  
QFormLayout Class  
application form GUI solution  
GUI application  
objects  
QSpinBox/QComboBox widgets  
setInputMask()  
setPrefix()  
QFrame container  
QFrame widgets  
QGraphicsItem.setPos() method  
QGridLayout layout manager  
definition  
to do list GUI  
to do list GUI solution  
ToDoList class  
todo\_title QLabel widget  
QGroupBox widget  
QHBoxLayout/QVBoxLayout Classes  
DisplaySurvey class  
GUI  
QButtonGroup class  
QHBoxLayout object  
styles  
survey GUI  
QLabel widget  
QLCDNumber widget  
QLinearGradient class  
QLineEdit widget  
QListWidget methods  
QMainWindow class  
QMdiArea class  
QMessageBox widget  
code  
dialog box  
GUI  
types  
windowsvs . dialogs  
QMimeData class  
QPainter class  
QPen class  
QPolygon class  
QProgressBar widget  
QPropertyAnimation class  
QPushButton widget

- code
- events
- QRadioButtons
- QToolButtons
- signals
- slots
- window
- QRadioButton widgets
- QSlider class
- QSqlRelationalTableModel class
- QSqlTableModel class
- QTableWidget class
- context menu
- functions
- table menu
- Qt Designer
- actions
- adding menus/submenus
- creating application
- definition
- editing tools
- functionality
- keypad GUI
  - See* Keypad GUI
- Main Window template
- multilevel layout
- QLabel widget
- toolbars
- user interface
- action editor
- form dialog box
- object inspector
- property editor dock widget
- resource browser
- widget box dock
- QTextEdit widget
- QtMultimediaWidgets module
- QToolBar class
- QTreeView class
- Qt style sheets
- individual widget properties
- technique
- QtWebEngine module
- QtWidgets module
- R**
- redValue() function
- Registration module
- Relational database management systems (RDBMS)
- repaint() method
- resizeEvent() event handler
- Resource Browser dock widget
- RGB slider
- adding methods
- colour update
- custom widget
- demo
- handling image data
- QSlider class
- QSlider/QSpinBox
- Rich Text Notepad GUI
- application

- design
- menubar/QTextEdit widget
- solution
- code
- open\_act object
- PyQt, images
- QDesktopWidget class
- QPrinter class
- QPrintSupport module
- QTextEdit widget
- QtGui module
- S**
- selectionChanged() signal
- self.close() method
- sender() method
- setAcceptDrops() method
- setAlternatingRowColors() methods
- setAutoExclusive() attribute
- setColumnCount() method
- setContentMargins() method
- setDefault() method
- setDragEnabled() method
- SetEchoMode() method
- setEnabled() method
- setFixedSize() method
- setHeaderData() method
- setIconSize() method
- setInputMask() method
- setItem() method
- setKeyValueAt() method
- setModel() method
- setMouseTracking() method
- setObjectName() method
- setPlaceholderText() method
- setRowCount() method
- setShortcut() method
- setSizePolicy() method
- setStopPoint() method
- setText() method
- setToolTip() method
- setupCamera() method
- setupTab() method
- setupWidgets() method
- setupWindows() method
- setWindowIcon() method
- setWindowTitle() method
- showPassword() function
- Signal/Slot Editor
- Simple clock GUI
- calendar date/clock time
- code
- Single-document interface (SDI)
- SIP binding generator
- Slider widgets
- Spin box widgets
- String data type
- immutable
- methods
- Structured Query Language (SQL)

- commands
- keywords/functions
- RDMBS
- T**
- tab\_bar widget
- Text editing classes
- text() method
- Thread
- definition
- GUI, file renaming
- directory, code
- QFileDialog
- QLineEdit widget
- QTextEdit/QProgressBar widgets
- QThread
- RenameFileGUI class
- parallelism
- timeout() signal
- toggled() signal
- toggle() method
- toString() method
- trigger() method
- triggered.connect()
- Typecasting
- U, V**
- Uniform Resource Locator (URL)
- User interface (UI)
- User profile GUI
- empty window, create
- code
- modifying
- operating system
- QApplication
- QtWidgets module
- layout design
- QLabel widgets
- solution, code
- user's personal data
- W, X, Y, Z**
- Web browser GUI
- backPageButton() slot
- code
- creating tabs
- features
- initializeUI()
- QLineEdit widget
- setupWebView() function
- updateProgressBar() slot
- URL
- Widget Box dock widget